

Python and DataFrames for Sensible Experiment Management

Ben Keene

University of Central Florida

23 June 2026

Overview

Structured, reproducible workflows for computational research.

Today we build a small LLM-benchmarking pipeline end to end, resting on one idea:

Dataframes as experiment managers:

- ▶ Make parameter sweeps, each run a row.
- ▶ Save entire sweep as CSV
- ▶ Load it back in, run each row, write results to disk.
- ▶ Collate results into a single dataframe for analysis.

The Question

- ▶ <https://arxiv.org/abs/2205.11916> — *Large Language Models are Zero-Shot Reasoners* (Kojima et al., 2022).
- ▶ The claim: simply appending **“Let’s think step by step”** to a prompt improves reasoning.
- ▶ Our job today: reproduce it. Test it against grade-school arithmetic problems.

Example Problem

Think back to 2022, before the explosion of LLMs.

Felix has 13 coins. Felix drops 10 coins. Felix drops 2 coins. Felix is given 1 more coin. How many coins does Felix have now?

This is not an easy question for a 2022 LLM to answer correctly.

We have two prompts: a bare standard prompt and `zs_cot` (“let’s think step by step”). We will compare accuracy between these strategies.

Reproducibility

One run proves nothing. To make a claim you'd defend, you need:

- ▶ **Many samples**, different questions.
- ▶ **Repeats**, different random seeds.
- ▶ **Fixed seeds**, for reproducibility (NB: LLM temperature).
- ▶ **Recorded work**, so you can re-run stages of the workflow.

That discipline *is* the pipeline. The rest of the talk is its five stages.

The Big Picture

Each script performs one function.

make_arithmetic.py	-> data/arithmetic.jsonl	the dataset
build_experiment.py	-> experiment.csv	PLAN 1 row / condition
runner.py	-> runs.csv	RUN 1 row / execution
	results/{run_id}.jsonl	+ per-sample answers
collate.py	-> collated.csv	COLLATE 1 row / question
v	plots	ANALYSIS

Stage 0: Data Generation

`make_arithmetic.py` writes `data/arithmetic.jsonl`. Each item is a dict: a question, the integer answer, the steps it took, and an `n_steps` count.

```
>>> dataset[0]
{'question': 'Mia has 12 coins. Mia drops 5 coins. '
             'Mia is given 3 more coins. '
             'How many coins does Mia have now?',
 'answer': 10,
 'n_steps': 2,
 'steps': [-5, 3]}
```

Seeded RNG = Reproducibility

The generator seeds Python's RNG from its seed argument.

What will the output be?

```
a = generate_arithmetic(n_steps=3, seed=42)
b = generate_arithmetic(n_steps=3, seed=42)

a == b      # ?
```

This is key: When you need to reproduce your plots, you can re-run the pipeline with the same seeds and get the same results.

Stage 1: PLAN

Construct a factory/registry of tasks and LLMs. Dict with keys as names, and values are `None` because we don't need to execute these tasks yet, only planning right now.

```
TASK_REGISTRY = {  
    "arithmetic": None,  
    # "coin_flips": None,  
}  
  
LLM_REGISTRY = {  
    "qwen2.5-7b": None,  
}
```

`build_experiment.py` raises a `KeyError` if a sweep references a task or LLM that isn't registered.

Stage 1: PLAN, Sweeps

Declare an arg space; take its Cartesian product. One *condition* is one point in that product.

```
TASK_SWEEPS = {
    "arithmetic": {
        "condition": ["standard", "zs_cot"],
        "n_samples": [10],
    },
}

LLM_SWEEP = {"llm": ["qwen2.5-7b"],
             "temperature": [0.0], "top_k": [3]}

llm_combos = itertools.product(
    *(LLM_SWEEP[k] for k in keys))
```

Stage 2: RUN, Expansion

The plan has one row per *condition*. Running it expands each condition into repeats executions, each with its own `run_id` and `seed`.

```
for _, cond in plan.iterrows():
    for k in range(int(cond["repeats"])):
        rows.append({
            "run_id": f"run{cond['idx']:02d}_rep{k}",
            "condition_idx": int(cond["idx"]),
            "repeat_idx": k,
            "seed": int(cond["base_seed"]) + k,
        })
```

That per-run seed is what makes a single execution reproducible.

Stage 2: RUN, The Live LLM

`runner.run_all` calls Qwen2.5-7B through `llm.py`, an OpenAI-compatible client that submits queries to our LLM through a parallelized `ThreadPoolExecutor`.

```
model = Qwen(temperature=0.0, top_k=3)

answers = model.chat([
    "Mia has 12 coins. ...",
    "Leo has 8 apples. ...",
]) # -> list of completions, one per prompt
```

Each run writes its samples to `results/{run_id}.jsonl` and a summary row to `runs.csv`.

Stage 3: Collate

`collate.py` reads the individual outputs of each run and combines them into a single dataframe.

```
frames = []
for _, run in runs.iterrows():
    with open(run["results_path"]) as f:
        samples = pd.DataFrame(
            json.loads(line) for line in f)
    for col in RUN_META:
        samples[col] = run[col]
    frames.append(samples)

df = pd.concat(frames, ignore_index=True)
df["correct"] = df["prediction"] == df["answer"]
```

Getting Started

Navigate to the following URL to get started:

<https://bit.ly/rcijupyter>.

Your username is the first letter of your first name, and your full last name.

John Doe $\mapsto$ jdoe

Ima Knight $\mapsto$ iknight

Remember your password!

Your password will be set to whatever you enter in the password field. Make sure to remember it for future use.

Analysis — Accuracy by Condition

One tidy dataframe, one `groupby`. This is the headline:

```
df.groupby("arg_condition")["correct"].mean()
```

Predict it first. Does `zs_cot` (“let’s think step by step”) actually beat the bare standard prompt?

We reveal the real numbers live in the notebook.

Analysis — Accuracy by Difficulty

`n_steps` is how many arithmetic operations the problem chains.

```
df.groupby("n_steps")["correct"].mean()
```

Predict it. Do you expect accuracy to fall as the problems get longer? By how much?

This column only exists because `collate` carried it through untouched.

Analysis — The Cost of Reasoning

zs_cot makes the model generate a whole chain of reasoning before answering. What does that do to wall-clock time?

```
(df.groupby(["arg_condition", "run_id"])
     ["runtime_s"].first()
     .groupby("arg_condition").mean())
```

The accuracy/cost tradeoff: if zs_cot wins on accuracy, what did it cost you per question?